Automatic Interlinear Glossing for Low-Resource Languages

Emery Jacobowitz
Brandeis University

Abstract

The generation of Interlinear Glossed Text (IGT) is an essential task in documenting a language's morphology. Many understudied languages lack extensive corpora of glossed text. I present a three-part system for the automatic generation of IGT from morphemes in three low-resource languages from the 2023 SIGMORPHON shared task on IGT: Tsez, Nyangbo, and Natügu. The basic component uses a fine-tuned ByT5 encoder with a Conditional Random Field head to predict glosses for morpheme sequences. The second component uses an LLM with few-shot learning to revise the predicted glosses. The third component is an active-learning system which can incorporate user feedback without re-training.

The PLM+CRF glosser displays highly-competent performance, achieving >90% morpheme accuracy in all three languages. However, the LLM components do not perform well, and likely contribute increased noise to the results.

1 Introduction

The generation of interlinear glossed text (IGT) is a core component of linguistic fieldwork. This task is critical in the description of a language's morphology, and may further be useful in demonstrating syntactic and phonological phenomena. The standard format for IGT is specified by the Leipzig Glossing Rules (Leipzig Glossing Rules), which provides a set of formatting guidelines and common abbreviations for linguistic terminology. Despite this, many authors use similar though distinct conventions which may be better-suited to individual languages or tasks.

The IGT for an arbitrary source sentence consists of at least three lines: surface form, morphemic gloss, and translation. The surface form exhibits a phonological form of the sentence, with hyphens marking morpheme boundaries. The morphemic gloss mirrors the boundaries of the first line, but contains a description of the meaning of each morpheme. The third line is a translation. It is common for authors to include more lines variously containing orthographic forms, literal translations, and bibliographic information. Common domain-specific terminology is often specified by a defined set of abbreviations rendered in (small) caps. A sample of IGT is given in Figure 1

(1) *ha* *mi* *ək* *eʲːəx.*
be.PRS.INDEP 1SG.NOM at shout.VN
'I am shouting.'
(lit.) 'I am at shouting.'

Figure 1: Sample interlinear glossing for a sentence in Scottish Gaelic, elicited and created by the author. Periods delimit distinct meanings conveyed by a single morpheme.

The creation of IGT is often most relevant for low-resource languages, where descriptive efforts are frequently incomplete and ongoing. In this paper, I describe a novel methodology for automatically generating IGT using an byte sequence-based pretrained language model (PLM) and conditional random field (CRF). I explore the use of a large language model (LLM) to improve these annotations. I also introduce an active learning system to incorporate human annotator feedback for generated IGT. The goal of this system is to provide a support tool for descriptive linguists working with low-resource languages where traditional computational approaches may be unhelpful due to limited corpora.

2 Related Work

Computational approaches to assist descriptive linguistic work are not new. (Baldridge and Palmer,

2009) used a maximum-entropy model to predict single morpheme glosses, finding that the system could improve annotation efficiency, albeit dependent on the expertise of the annotator. This represented an early application of *active learning*, where annotator feedback is used to update a predictive model as the work is carried out. (Samardžić et al., 2015) separately predicted lexical and grammatical meanings before combining them in IGT, with the former requiring a pre-defined lexicon. (McMillan-Major, 2020) further applied statistical modeling, demonstrating decent accuracy with a CRF.

Modern approaches are demonstrated by (Zhao et al., 2020) and (Ginn et al., 2024), who both used Transformer models—the latter achieving high multilingual accuracy using a byte-sequence model. Interest in IGT generation increased as a result of the 2023 SIGMORPHON shared task (Ginn et al., 2023). Recently, (Liang et al., 2026) have shown the effectiveness of using an LLM to improve prediction quality of BiLSTM-based prediction. (Elsner and Liu, 2025) explored a prompting-only LLM approach, achieving promising results though with the caveat of prohibitive computational and monetary costs.

3 Data

I used the data provided by the 2023 SIGMORPHON Shared Task on Interlinear Glossing (Ginn et al., 2023)[1]. This dataset includes IGT from seven low-resource languages on two tracks: The "closed" track where only the bare text is provided, and the "open" track where pre-segmented morphemes are given.

3.1 Language Descriptions

I selected data from three languages to work with:

1. **Tsez** (ddo), Northeast Caucasian. Spoken in southwestern Dagestan, Russia.
2. **Nyangbo** (nyb), Kwa-Togo. Spoken in the highlands of northern Ghana and Togo.
3. **Natügu** (ntu), Oceanic. Spoken on Nendö island in the Solomon Islands.

These languages were chosen due to the amount of data available in the corpus, as well as the quality of their available glosses. Sample IGT from Tsez

[1]Available at https://github.com/sigmorphon/2023glossingST

```
\t K'uruč' k'eč' ɣudi rocu, buq bay!
\m k'uruč' k'eč' ɣudi r-ocu-u buq b-ay
\g Kuruch pearl day IV-become.clean-IMPR sun III-come
\l All's well that ends well.
```

Figure 2: A gold Tsez sentence from the SIGMORPHON '23 dataset. \t is the surface text, \m is the segmented morphemes, \g is the morpheme-by-morpheme gloss, and \l is the English translation.

is given in Figure 2. The total number of sentences per language per split is given in Table 1.

Language	# Train	# Dev	# Test
Tsez	3,558	445	445
Nyangbo	2,100	263	263
Natügu	791	99	99

Table 1: The number of IGT sentences per language per split, SIGMORPHON dataset.

3.2 Track Selection

The issue of segmenting morphemes is non-trivial. This may be surprising to speakers of some languages—isolating languages like Vietnamese have close to one morpheme per word, so morpheme segmentation lines up naturally with word boundaries. In other languages like Turkish, morpheme boundaries are frequently distinct from the surface text, so there is a clear location for each break.

However, this is not the case cross-linguistically. Many languages have morphophonological processes that alter the surface forms of morphemes to the point that there is clear alignment from surface to underlying form. Furthermore, morphemes with no phonological value–termed *null morphemes*–do not correspond to the surface at all. An example of a glossed Tsez word is given in Figure 3, demonstrating the challenge in this process.

(2) *esnazar*
esyu-bi-r
brother-PL-LAT

'toward the brothers'

Figure 3: An example Tsez word where the surface form does not neatly map to morpheme boundaries as a result of phonological processes.

As a result of this challenge, my focus is on the "open" track, where a gold morpheme segmentation is assumed. This makes the system less powerful for descriptive use, as it requires an expert annotator to perform the segmentation ahead of time. However, this tradeoff allows the system to achieve much stronger performance than comparable systems achieve on "closed" data.

4 Methods

The system is divided into three parts: the **glosser**, which predicts IGT from morphemes, the **reviser**, which attempts to improve predicted glosses, and the **active learning setup**.

4.1 Glosser

The glosser consists of two parts: an encoder and a CRF. The encoder model is ByT5 (Xue et al., 2022), a byte-sequence model based on the T5 architecture. The use of a byte-sequence model is significant. Traditional token-based pretrained models (PLMs) like BERT (Devlin et al., 2019) include a pretrained tokenizer and vocabulary. This poses an issue for low-resource languages, which are not sufficiently represented in the training set. Using a pretrained tokenizer is likely to introduce errors by either failing to interpret terms from the target language, or worse, confusing them with terms from other languages. In some scenarios, such as when a genetically related language is well-represented, this can be a benefit. However, for the target languages of this study, BERT-esque models are not optimal.

A bytes-based model also has the benefit of being orthography-agnostic, being able to learn any encoding of characters. This is useful for some of the languages in this task, which use uncommon scripts that could pose an issue for systems not expecting the full range of Unicode codepoints.

A tradeoff of the ByT5 model, compared to other comparable transformer models, is memory. Byte-sequences tend to be significantly longer than token-sequences representing the same text. This specifically poses an issue for self-attention, where time and memory complexity are quadratic based on the maximum sequence length. In practice, this meant that the ByT5 model necessitated a smaller batch size when training to avoid overshooting memory limitations.

Formally, the glosser takes as input gold morphemes **m** and gloss **g**. For each byte in morpheme m_t, the encoder projects a hidden state with dimensionality d_{enc}. Those hidden states are averaged into the vector $\mathbf{h}_t \in \mathbb{R}^{d_{enc}}$.

A linear projection P with hidden dimensionality d_p is applied to the hidden states, and then a nonlinearity function. A dropout layer is applied, and finally a classifier network W transforms the vectors into $|V|$-dimensional logits, where V is the gloss vocabulary size.

$$\mathbf{u}_t = W \cdot \text{GELU}(P\mathbf{h}_t) \in \mathbb{R}^{|V|}$$

After this, the role of the ByT5 encoder is completed. $\mathbf{u}_t$ is padded and masked to ensure a uniform sequence length, before being fed into the CRF as emission scores. The CRF returns the conditional log likelihood of **g** given the emission scores; this value is multiplied by -1 and represents loss. The Viterbi algorithm is used to predict the most likely gloss sequence based on the emissions.

The CRF is useful to help compensate for some of the PLM's shortcomings. While the PLM is effective at capturing overall sentence structure, the CRF is specialized in capturing transition probabilities between morphemes. This is very effective for learning the distribution of affixes, for example. The CRF also ensures that the length of the predicted gloss match the length of the input sequence, and guarantee not enforced by encoder-decoder models.

4.2 Glosser Training

Three glossers were trained in Google Colab using a T4 GPU using hyperparameters given in Table 2. Accumulated gradient steps refers to the number of forward passes before a backward pass, which approximates a larger batch size without the memory overhead. Note that early stopping was employed, and all three models terminated training after about 30-40 epochs.

Due to compute limitations, a full hyperparameter exploration was not explored.

4.3 Reviser

The reviser consists most broadly of an existing LLM which is prompted with the glosser's prediction. Importantly, the reviser *has not* been fine-tuned on language-specific data. Its task is simply to apply its existing reasoning abilities, not domain knowledge.

The reviser is instructed to attempt to improve the gloss if possible, based on *k* examples. These

Hyperparameter	Value
Model	`ByT5/small`
Trainable Parameters	300M
Batch size	4
Accumulated gradient steps	4
Encoder learning rate	2×10^{-5}
CRF learning rate	1×10^{-3}
Maximum epochs	50
Projection dim.	256
Dropout probability	0.3

Table 2: Caption

examples are retrieved from the training set at runtime. During setup, a TF-IDF vectorizer is fit to the morpheme lines (`\m`) of the train set. At inference time, the target sentence's morpheme line is vectorized. Then, the k most similar examples are retrieved using the k-nearest neighbors algorithm. These examples are included in the prompt for in-context learning. Finally, the prompt is given to the LLM, and it returns its improvement attempt.

The primary model used for the retriever was Google's `gemma-4-31b-it`, which was accessed through a remote API. Smaller models–`Llama3.2-1B-instruct` and `Qwen-2.5-1B-instruct` were tested locally, though these models rarely succeed in creating a well-formed output, so they were discontinued.

A k value of 10 was used when prompting for Tzes and Nyangbo. To compensate for the smaller corpus size of Natügu, k was set to 5.

4.4 Active Learning

The active learning system was a fairly natural extension of the base reviser. It involves two components: A feedback store where user-corrected examples are kept, and comparison system to keep track of whether the glosser or the reviser (or both) produced preferable output.

During use, the sentences in the feedback store are also vectorized for retrieval. These examples are weighted more heavily than comparable examples from the original training pool.

A command-line interface (CLI) was implemented to allow the user to view an example and both predicted glosses. They can then choose to accept one of the proposed glosses, edit it, reject it entirely, or skip the example. Accepted glosses are then added to the feedback store.

4.5 Evaluation

Evaluation of the glosser, reviser, and active learning system were all handled separately.

There are two primary metrics to evaluate gloss quality (Ginn et al., 2023): word-level accuracy and morpheme-level accuracy. Morpheme accuracy simply represents the percentage of correct morphemes:

$$m_{acc} = \frac{\text{Correct morphemes}}{\text{Total morphemes}}$$

Word accuracy measures the same, but on the word level. A word is correct if and only if all of its morphemes are correct.

$$w_{acc} = \frac{\text{Correct words}}{\text{Total words}}$$

Word accuracy will therefore always be lower than morpheme accuracy, unless they both equal 1.0. Morpheme accuracy is useful for getting a picture of overall system performance. Word accuracy is useful in determining if errors are widespread or constrained to a smaller subset of words.

4.6 Roundtrip Evaluation

I introduce a novel metric for measuring gloss accuracy, *roundtrip performance*. In a roundtrip test, an LLM call is used to "translate" the both the gold and predicted glosses English sentence. These translations are compared to the original translation using a character F1 (chrF) to measure string similarity.

The chrF of the translation of the gold gloss serves as a point of reference for the translation of the predicted gloss. The gloss contained in IGT conveys grammatical and lexical meaning, but it does not provide a complete understanding of the sentence. This metric is not useful for determining the accuracy of the prediction, rather it is useful in approximating how much of the core meaning was preserved in the predicted gloss.

5 Results

Table 3 reports the test set performance of the base glosser compared to the baseline RoBERTa system and best-performing model from SIGMORPHON in terms of morpheme accuracy. Table 3 reports the same results in terms of word accuracy.

The final column of both tables describes the performance of the reviser on the same test set as the glosser. This result requires a caveat—due to

issues with the LLM API[2], this score is based on only a smaller portion of the test set. Therefore these scores cannot be directly compared to the other results.

In individual cases, the reviser often performed *worse* than the glosser, as discussed in Section 6.

My approach seems to be on-par with the best results from the workshop, without accounting for variation between evaluation runs. These results are incredibly promising—they demonstrate that SOTA performance is possible with a relatively simple system. Even the reviser is not necessary to achieve these scores.

5.1 Roundtrip Scores

Due to the same API limitations mentioned above, roundtrip similarity scores were incredibly difficult to measure. Each evaluation takes four API calls: one to revise the predicted gloss, two to translate the predicted glosses, and one to translate the gold gloss. The Gemma API reutrned a `500` error approximately 25% of the time—proving a real issue for this strategy.

An entirely unscientific approximation of this evaluation strategy is demonstrated with a representative example in Table 5.

5.2 Active Learning Evaluation

The active learning system is very difficult to evaluate quantitatively because its goal is not to maximize performance, but to learn from feedback. To evaluate this, I attempted to insert noticeably incorrect glosses, which would not be present in the glosser vocabulary. If the system is able to learn from feedback, it should start to propagate those errors into future examples.

I worked through a short session of 10 Tsez examples. I edited every one of the examples, replacing instances of the `CVB` converb morpheme (an auxiliary particle associated with many verbs) with the gloss `HAMBURGER`. Unfortunately, 10 examples was not enough to make a change in the system; I never observed a revised gloss including the `HAMBURGER` morpheme.

6 Discussion

The results of the system seem to be divided by architecture. The PLM- and CRF-based glosser performed very well, matching SOTA results in peer systems. Overall the LLM-based reviser and active learning system performed poorly.

Part of this can be explained by material limitations. There was insufficient time and budget to do a full exploration of LLM capabilities, and the present experiments were hampered by API restrictions. Current research suggests LLMs can do a very good job at this task, so this is an important avenue for future work.

For the poor qualitative performance of the active learning system, the blame likely rests on the example retrieval system, not the LLM itself. TF-IDF vectorization gives more weight to features that are common in a document but rare in the corpus. Treating morphemes as features, the grammatical morphemes are going to be much more frequent across documents than lexical morpheme, so they will be weighted down. Thus, the retrieval system will be biased in favor of examples that discuss similar topics, not necessarily ones that have similar morphological structure.

6.1 Reviser Shortcomings

However, there is likely more to the story of why the LLM reviser is unable to consistently improve accuracy scores. Even though the glosser's output is present in the reviser's input, the reviser frequently *lowers* accuracy.

I suspect this is the result of a "Catch-22" of sorts. The glosser is powerful enough to handle most sentences, leaving only difficult cases. These difficult cases are why the LLM is motivated to begin with, as its strengths lie in synthesizing from direct examples. However, it is likely that these cases are *too* hard, e.g. unexplained grammatical irregularities, or out-of-vocabulary terms that don't match the existing patterns, such as loanwords. In these cases, the LLM's expressiveness may actually harm its accuracy by introducing too much noise into the process.

Furthermore, morphological glossing is by no means a solved problem for human annotators. The same sentence can be glossed in multiple ways depending on the level of analysis (broad vs. narrow), the focus of the annotations (e.g. morphophonology vs. morphosyntax), or even the analytic framework being applied. These issues are even more pronounced for low-resource languages, some of which are described solely by a single linguist.

[2]Google is arbitrary, unreliable, and capricious with rate limits and `500` errors.

Language	Baseline	Best	Glosser	Reviser*
Tsez	85.34	92.01	**92.45**	93.75
Nyangbo	88.71	91.40	**91.60**	89.31
Natügu	49.03	92.77	**92.82**	91.69

Table 3: Morpheme-level accuracy (%) of the glosser against the SIGMORPHON 2023 shared-task baseline and best results. Note that Reviser scores are artificially inflated due to API issues.

Language	Baseline	Best	Glosser	Reviser*
Tsez	75.71	85.79	**86.01**	87.56
Nyangbo	84.30	87.98	**88.13**	84.09
Natügu	41.08	**89.31**	88.85	86.50

Table 4: Morpheme-level accuracy (%) of the glosser against the SIGMORPHON 2023 shared-task baseline and best results. Note that Reviser scores are artificially inflated due to API issues.

Translation	chrF vs. Gold
Glosser	42.12
Reviser	37.54
Gold gloss	44.00

Table 5: Rountrip testing on a single Tsez example. Due to LLM API limitations, roundtrip scoring is not a feasible option.

6.2 A Positive Outlook

While the LLM components of this system are unlikely to see field use, the PLM+CRF glosser is incredibly useful. The fact that it was able to achieve >90% morpheme accuracy on datasets as small as 700 sentences is of great import for descriptive linguists. Having a reasonable prediction for a gloss before you even look at it could drastically speed up annotation rates, facilitating the collection of more data, and allowing the system to be improved faster. This is a valuable process, even if it is not true active learning.

7 Conclusion

I have developed a system for the generation of interlinear glossed text from pre-segmented morphemes. The base glosser uses ByT5 as an encoder with a CRF head, achieving >90% morpheme accuracy on data in three low-resource languages. The reviser component uses an LLM with in-context learning based on similar examples, which has had mixed results. The reviser allows the creation of an active learning system to improve performance without retraining, though it has not been demonstrated to significantly improve prediction quality over time.

Future work could involve more detailed hyperparameter exploration for the glosser and comparison with other byte-sequence models, as well as prompt exploration for the reviser. More generally, IGT generation is a relatively understudied field with ample room for future research. Future systems could work on developing multilingual models, or on different aspects of the IGT process, such as generating a natural-language translation from glossed morphemes.

References

Anthropic. 2026. System card: Claude opus 4.7. Technical report, Anthropic.

Jason Baldridge and Alexis Palmer. 2009. How well does active learning *actually* work? Time-based evaluation of cost-reduction strategies for language documentation. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pages 296–305, Singapore. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Micha Elsner and David Liu. 2025. Prompt and circumstance: A word-by-word LLM prompting approach to interlinear glossing for low-resource languages. In Proceedings of the 22nd SIGMORPHON workshop

on Computational Morphology, Phonology, and Phonetics, pages 1–14, Albuquerque, New Mexico, USA. Association for Computational Linguistics.

Michael Ginn, Sarah Moeller, Alexis Palmer, Anna Stacey, Garrett Nicolai, Mans Hulden, and Miikka Silfverberg. 2023. Findings of the SIGMORPHON 2023 shared task on interlinear glossing. In Proceedings of the 20th SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology, pages 186–201, Toronto, Canada. Association for Computational Linguistics.

Michael Ginn, Lindia Tjuatja, Taiqi He, Enora Rice, Graham Neubig, Alexis Palmer, and Lori Levin. 2024. GlossLM: A massively multilingual corpus and pretrained model for interlinear glossed text. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 12267–12286, Miami, Florida, USA. Association for Computational Linguistics.

Leipzig Glossing Rules. 2015. Leipzig Glossing Rules.

Siyu Liang, Talant Mawkanuli, and Gina-Anne Levow. 2026. Hybrid neural-LLM pipeline for morphological glossing in endangered language documentation: A case study of jungar tuvan. In Proceedings of the Fifth Workshop on NLP Applications to Field Linguistics, pages 16–30, Rabat, Morocco. Association for Computational Linguistics.

Angelina McMillan-Major. 2020. Automating gloss generation in interlinear glossed text. In Proceedings of the Society for Computation in Linguistics 2020, pages 355–366, New York, New York. Association for Computational Linguistics.

Tanja Samardžić, Robert Schikowski, and Sabine Stoll. 2015. Automatic interlinear glossing as two-level sequence classification. In Proceedings of the 9th SIGHUM Workshop on Language Technology for Cultural Heritage, Social Sciences, and Humanities (LaTeCH), pages 68–72, Beijing, China. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. Transactions of the Association for Computational Linguistics, 10:291–306.

Xingyuan Zhao, Satoru Ozaki, Antonios Anastasopoulos, Graham Neubig, and Lori Levin. 2020. Automatic interlinear glossing for under-resourced languages leveraging translations. In Proceedings of the 28th International Conference on Computational Linguistics, pages 5397–5408, Barcelona, Spain (Online). International Committee on Computational Linguistics.

A LLM Use Statement

The majority of the code I have submitted was generated with Claude Opus 4.7 (Anthropic, 2026). I ended up using about $120 worth of Claude credits allotted to me by my undergraduate institution, as my alumni access is supposed to be cut off any day now. All high-level instruction was given by me, such as the models and libraries to use, the system architecture, which design decisions to make, and so on. Most of the low-level implementation details were generated, though I of course crawled through all of them to check for errors. This writeup was completely created by me, except for the figures which were programmatically generated.

This was my first time using a coding assistant on a medium-scale project. I was pleasantly surprised by how capable Claude was, though of course there would have been a significant monetary cost. I suspect that Claude's success was due in part to the fact that this project doesn't implement anything truly novel in terms of architecture. I consider this a successful experiment in learning how to use an AI coding assistant.

B LLM Reviser Prompts

System prompt:

```
You are a linguistic expert specializing in
    morpheme-by-morpheme glossing for an unknown
     language. You will be given example
    sentences with their morpheme segmentation (
    M:), gloss (G:), and free translation (T:).
    Your task is to produce a gloss for a new
    test sentence. Give concise answers.
```

Output instructions:

```
Output ONLY the gloss line, hyphen-separated
    within words and whitespace-separated
    between words. Wrap the gloss in delimiters:
     #### ... ####. The number of hyphen-
    separated tokens in your gloss must match
    the number of morphemes in the test sentence
    .
```

Revision prompt:

```
Here are example glossed sentences from this
    language:
{examples_block}
---
Now correct any errors in the rough initial
    gloss for this sentence.
You may keep parts that look right.
{test_block}
```

Translation prompt:

```
Here are example glossed sentences from this
    language:
{examples_block}
---
```

```
Now produce a gloss for this sentence:
{test_block}”
```